

From Bits to Trust: AI-Driven Reconfigurable Security

Rye Stahle-Smith

Dept. of Computer Science and Engineering
University of South Carolina
Columbia, SC, USA
ryes@email.sc.edu

Rasha Karakchi

Dept. of Computer Science and Engineering
University of South Carolina
Columbia, SC, USA
karakchi@cec.sc.edu

Abstract—Reconfigurable computing platforms such as Field-Programmable Gate Arrays (FPGAs) are widely deployed in embedded systems and AI acceleration workloads. Their reconfigurability, however, also introduces security risks when hardware behavior is defined by externally supplied configuration bitstreams. Maliciously crafted bitstreams can embed hardware Trojans capable of leaking data, degrading system reliability, or altering computation.

This paper presents BLADEI (*Bitstream-Level Anomaly Detection for Embedded Inference*), an AI-based predictive modeling framework for detecting malicious FPGA configurations directly from binary bitstreams. A 278-dimensional feature vector combining byte-frequency histograms, statistical summaries, and structural indicators is extracted from each bitstream and classified using supervised machine learning models. The proposed approach is evaluated on an expanded dataset of 1,090 bitstreams spanning thirteen design families re-engineered for the Xilinx PYNQ-Z1 platform, augmented through Vivado place and route directives to improve dataset diversity. Experimental results demonstrate that a Random Forest classifier achieves a macro F1-score of 0.91 on held-out test data while remaining feasible for on-device deployment, validating the effectiveness of AI-driven bitstream screening in resource-constrained environments.

Index Terms—FPGA security, hardware trojan, artificial intelligence, predictive modeling, bitstream analysis, PYNQ

I. INTRODUCTION

Field-Programmable Gate Arrays (FPGAs) are reconfigurable semiconductor devices that enable hardware functionality to be defined and updated after manufacturing through binary configuration files known as bitstreams [1]–[5].

Unlike fixed-function processors or application-specific integrated circuits (ASICs), FPGAs expose a programmable hardware fabric composed of configurable logic blocks (CLBs) built from lookup tables (LUTs) and flip-flops, block RAMs (BRAMs), DSP slices, and a rich programmable interconnect network. As illustrated in Fig. 1, a bitstream specifies how these resources are instantiated and interconnected, effectively transforming the device into a custom hardware accelerator tailored to a given workload. At a low level, vendor-specific bitstream formats map configuration frames to physical rows and columns of the device; these frames are written into configuration registers that define routing paths, logic behavior,

memory contents, and clocking structures once configuration is applied [6], [7].

The ability to reconfigure hardware post-deployment has made FPGAs central to modern reconfigurable computing systems. They are widely used in embedded and real-time platforms for control and signal processing, in edge and cloud environments for network acceleration and encryption, and increasingly in artificial intelligence (AI) systems to accelerate inference with low latency and high energy efficiency [8]–[10]. Contemporary FPGA platforms further support partial and dynamic reconfiguration, allowing subsets of the fabric to be updated while other regions continue executing. This capability enables time-multiplexed accelerators, adaptive systems, and multi-tenant deployments in which hardware functions are loaded, swapped, or specialized at runtime [?], [8], [11].

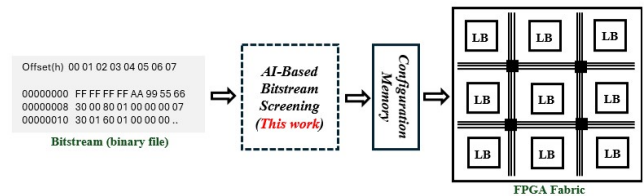


Fig. 1. Conceptual view of FPGA configuration via bitstreams. The binary configuration bitstream programs configuration memory, which defines logic blocks and routing resources in the FPGA fabric. The proposed AI-based bitstream screening operates prior to configuration, enabling detection of malicious payloads before hardware behavior is instantiated.

While reconfigurability provides significant performance and flexibility benefits, it also introduces a critical low-level attack surface. Because a bitstream directly programs physical resources on the device, a malicious or tampered bitstream can insert additional logic, alter routing to establish covert communication channels, modify finite-state machine triggers, or reconfigure memory resources to leak sensitive data. In practice, adversaries can implement a wide range of hardware Trojans, including always-on payloads that silently exfiltrate information, trigger-activated payloads that execute only under rare conditions, and resource-exhaustion payloads that induce denial-of-service by monopolizing logic or BRAM resources [6], [12], [13]. In AI-enabled systems, such attacks may subtly corrupt inference results, manipulate feature paths, or exfiltrate

This work was supported by the McNair Junior Fellowship and the Office of Undergraduate Research at the University of South Carolina.

trained model parameters processed in edge and embedded pipelines [9], [10].

These threats are amplified in multi-tenant cloud and shared-edge environments, where bitstreams are uploaded dynamically and deployed onto shared physical FPGA fabrics using time- or region-based multiplexing [14]. In such settings, tenants typically lack access to low-level hardware instrumentation or golden reference devices, and bitstreams may be generated by untrusted or partially trusted toolchains. Traditional defenses—including exhaustive design-time verification, side-channel analysis based on golden chips, or invasive laboratory measurements—are often infeasible due to privacy concerns, operational cost, and the proprietary nature of FPGA design tools [13], [15]. Reverse-engineering approaches that attempt to reconstruct netlists or structural representations from bitstreams can provide valuable insight, but they are computationally expensive, vendor dependent, and unsuitable for real-time or pre-deployment screening [7], [16], [17]. Moreover, recent demonstrations of automated Trojan insertion and toolchain compromise show that malicious modifications can be introduced at multiple stages of the design flow and may be highly obfuscated, further weakening assumptions of trustworthy synthesis and implementation [15], [18], [19].

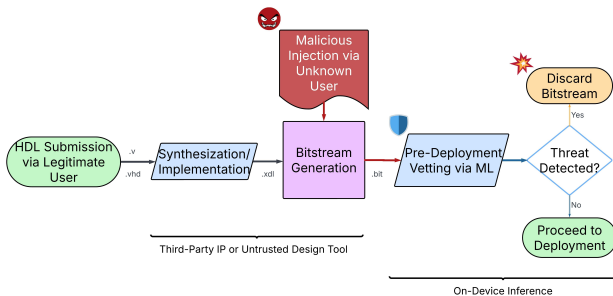


Fig. 2. Threat Model for Detecting Trojan-Injected FPGA Bitstreams in Multi-Tenant Systems.

Fig. 2 summarizes this operational threat model: untrusted configuration payloads are supplied to a reconfigurable FPGA platform in environments where per-tenant instrumentation and golden references are impractical. These constraints motivate a detection paradigm that treats the bitstream itself as the primary—and often the only—observable artifact available for security verification. In reconfigurable systems where bitstreams are loaded repeatedly and dynamically, security checks must therefore integrate directly into the reconfiguration workflow rather than relying solely on offline design-time analysis.

In this work, we present BLADEI (*Bitstream-Level Anomaly Detection for Embedded Inference*), an AI-based predictive modeling approach that operates directly on raw bitstreams and is explicitly designed for reconfigurable architectures. By viewing bitstreams as high-dimensional data objects, machine learning models can learn distributional and structural signatures that differentiate benign and malicious configurations without attempting full semantic reconstruction into netlists. To ensure robustness under realistic reconfiguration scenarios,

we evaluate the approach on a large, augmented dataset of 1,090 bitstreams spanning thirteen design families, generated by exploring multiple place-and-route implementations per design. This captures implementation-level variability inherent to reconfigurable workflows. The resulting framework enables scalable, pre-deployment screening of bitstreams and is well suited for embedded, edge, and multi-tenant FPGA platforms where fast, autonomous, and non-invasive verification is essential [20]–[22].

II. BACKGROUND AND RELATED WORK

FPGA configuration bitstreams encode the complete hardware state of a device—including lookup tables (LUTs), routing resources, memory blocks, and clocking structures—using vendor-specific and largely proprietary formats [6], [7]. While this representation enables efficient post-deployment reconfiguration, it is inherently opaque: bitstreams expose limited semantic insight into design intent, and reconstructing higher-level representations such as netlists is complex, incomplete, and strongly vendor dependent. As a result, traditional inspection techniques are difficult to apply and motivate detection approaches that operate without full structural reconstruction.

Hardware Trojans may be introduced at multiple stages of the FPGA lifecycle. At design time, malicious logic can be embedded in RTL or gate-level descriptions, while during synthesis and implementation compromised CAD tools can inject stealthy modifications that evade functional verification and equivalence checking [15]. In the most challenging threat model, Trojans are inserted directly into post-routing bitstreams, bypassing all design-time safeguards and leaving no trace in source code or intermediate artifacts [6], [13]. These multi-stage insertion vectors weaken defenses that assume access to trusted development artifacts.

Existing detection techniques can be broadly categorized by the artifacts and assumptions they require. Design-time verification, structural analysis, and benchmark-driven evaluation are effective when trusted RTL, netlists, or reference designs are available [12], [23], [24]. Side-channel approaches exploit power, electromagnetic, or timing signatures to detect anomalies, but typically rely on golden reference devices, controlled environments, and specialized instrumentation, limiting practicality outside laboratory settings [13]. Reverse-engineering techniques aim to recover structural representations from bitstreams to enable deeper inspection; however, they are computationally expensive, vendor specific, and unsuitable for real-time or embedded deployment [7], [16], [17].

Machine learning has therefore emerged as an alternative for detecting malicious FPGA configurations. Supervised classifiers trained on bitstream-derived features have shown strong performance on structured Trojan benchmarks, indicating that malicious modifications often introduce detectable statistical patterns [20], [25]. Sequence- and NLP-inspired approaches model bitstreams as ordered data streams to capture syntactic or temporal regularities associated with Trojan insertion [26], [27]. When reliable structural representations are available, graph neural networks and explainable ML techniques enable

fine-grained localization of malicious LUTs and routing resources [28], [29]. More recently, golden-free and unsupervised approaches have been proposed to reduce dependence on labeled malicious samples in deployed environments [22].

Despite these advances, most prior work assumes access to trusted intermediate artifacts, incurs substantial computational overhead, or targets server-class analysis environments. These assumptions limit applicability in embedded systems, edge deployments, and multi-tenant FPGA platforms, where verification must be fast, autonomous, and operate solely on configuration data available at deployment time [21]. In contrast, the approach proposed in this work is explicitly designed to function without source code, netlists, or trusted toolchains and to remain practical for resource-constrained platforms. The following section describes how this design goal is realized through an AI-driven, bitstream-only detection framework.

III. AI-BASED DETECTION APPROACH

Fig. 3 illustrates the complete lifecycle of the proposed bitstream-level Trojan detection framework, spanning offline model training and validation through on-device deployment and runtime inference. The workflow is intentionally divided into two phases. The offline phase, executed on a host workstation, is responsible for feature extraction, representation learning, model selection, and validation. The online phase executes on the target embedded platform (PYNQ-Z1) and performs autonomous screening of incoming bitstreams prior to deployment. This separation enables computationally intensive model training to be performed once, while keeping runtime inference lightweight and suitable for resource-constrained systems.

A. Feature Extraction

The first stage of the pipeline operates directly on raw FPGA configuration bitstreams without requiring source code, netlists, or trusted toolchain artifacts. Each bitstream file is treated as a binary sequence

$$B = [b_1, b_2, \dots, b_L],$$

where L denotes the total number of bytes and each byte value $b_i \in \{0, \dots, 255\}$. Rather than attempting to recover structural or semantic information from the bitstream, the approach summarizes its global statistical properties through a normalized byte-frequency histogram:

$$v_j = \frac{1}{L} \sum_{i=1}^L \mathbf{1}\{b_i = j\}, \quad j = 0, \dots, 255.$$

This results in a fixed-length, 256-dimensional feature vector v whose components represent the relative frequency of each byte value in the configuration file. This representation is vendor-agnostic, compact, and inexpensive to compute, making it well suited for embedded deployment. Importantly, even small modifications to routing, LUT configuration, or control frames often introduce measurable shifts in the byte

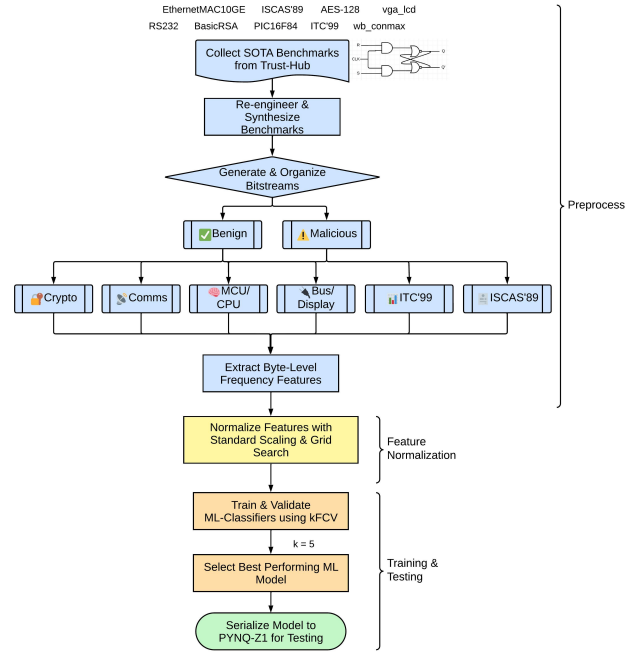


Fig. 3. Training and Deployment Pipeline for Bitstream-Level Trojan Detection.

distribution, allowing malicious alterations to be detected indirectly through statistical deviation rather than explicit structural reconstruction. Similar observations have been reported in prior bitstream-level machine learning studies [20].

To capture limited local ordering information, we additionally explored short byte n -gram features ($n = 2, 3$) using hashed buckets to control dimensionality. These sequence-inspired features were included only when they improved validation performance, consistent with recent work applying NLP-style representations to configuration data [26], [27]. However, the primary deployed pipeline relies on the simpler byte-frequency representation to minimize runtime overhead.

B. Dimensionality Reduction using TSVD

While the byte-frequency histogram provides a compact summary, the resulting 256-dimensional space still contains redundancy and noise that can negatively impact learning, particularly on small datasets. To address this, the pipeline applies Truncated Singular Value Decomposition (TSVD) to the matrix of feature vectors extracted from the training set [30]–[32]. TSVD identifies dominant modes of variation across bitstreams and projects each sample into a lower-dimensional latent space that preserves the most informative structure.

In practice, TSVD transforms each histogram vector into a k -dimensional embedding, where $k \ll 256$. The number of components is selected empirically to balance detection performance and computational cost; in our experiments, $k = 32$ provided a favorable trade-off between variance retention and embedded inference efficiency. Dimensionality reduction not only reduces memory footprint and prediction latency, but

also improves classifier robustness by suppressing noise and spurious correlations.

C. Class Balancing with SMOTE

FPGA Trojan datasets, both benchmark-based and real-world, are typically imbalanced, with benign configurations significantly outnumbering malicious variants. Training directly on such data can bias classifiers toward majority classes and reduce sensitivity to rare Trojan types. To mitigate this effect, we apply the Synthetic Minority Over-sampling Technique (SMOTE) in the TSVD embedding space during training [33], [34].

SMOTE generates synthetic minority samples by interpolating between nearest neighbors, effectively expanding the decision region associated with underrepresented Trojan classes. Importantly, oversampling is applied only within training folds during cross-validation, ensuring that no synthetic data leaks into evaluation sets. This step improves recall for minority Trojan classes while preserving realistic decision boundaries.

D. Model Selection and Validation

Following feature extraction, dimensionality reduction, and class balancing, multiple supervised learning models are evaluated to identify a classifier that offers strong detection performance and robustness to noisy, high-dimensional data. The evaluated models include Random Forests, Gradient Boosting, AdaBoost, Support Vector Machines (RBF kernel), k-Nearest Neighbors, Decision Trees, and Naive Bayes classifiers.

Model hyperparameters are tuned using grid search with stratified k -fold cross-validation [31], [35], ensuring that class proportions are preserved across folds. Among the evaluated models, Random Forest classifiers consistently demonstrated strong performance and stability, owing to their ensemble structure and resistance to overfitting in the presence of correlated or partially redundant features [21]. The best-performing model is selected for deployment.

E. Deployment on PYNQ-Z1

Once trained and validated, the complete preprocessing and inference pipeline, including feature normalization, TSVD components, and the trained classifiers are serialized and transferred to the target embedded platform. In this work, deployment is performed on the Xilinx PYNQ-Z1 board, which combines ARM processing cores with FPGA fabric and supports Python-based application development [11], [36]–[38]. At runtime, incoming bitstreams are processed locally on the device using the same feature extraction and transformation steps defined during training. This enables autonomous, pre-deployment screening of configuration files without reliance on external servers or cloud-based analysis. Fig. 4 illustrates the runtime inference pipeline executed on the PYNQ-Z1 through a Python-based interface.

IV. DATASET CONSTRUCTION AND AUGMENTATION

To construct a robust evaluation dataset, we sourced Verilog and VHDL benchmarks from Trust-Hub [19], focusing

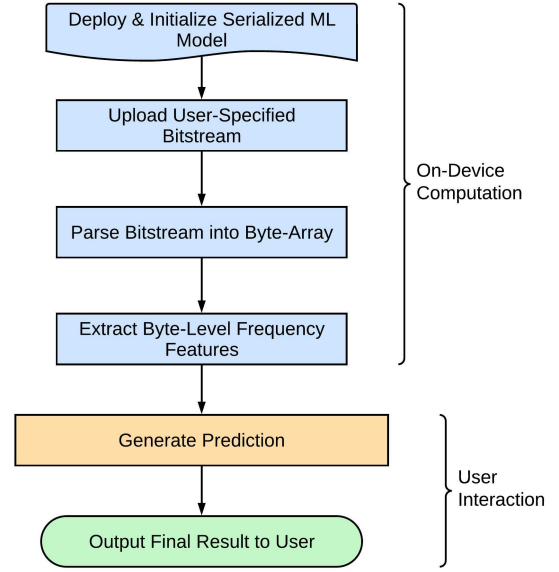


Fig. 4. Runtime Prediction Pipeline on PYNQ-Z1.

on chip-level (structured) Trojan designs. Each benchmark was re-engineered to target the Xilinx PYNQ-Z1 platform and synthesized, implemented, and bitstream-generated using Vivado v2023.2. The initial dataset consisted of 218 bitstreams across thirteen design families: AES (50), RS232 (50), EthernetMAC10GE (16), BasicRSA (8), PIC16F84 (8), wb_conmax (6), b15 (8), b19 (50), s15850 (2), s35932 (6), s38417 (6), s38584 (6), and vga_lcd (2). To augment the dataset while maintaining design integrity and preserving Trojan semantics, we leveraged Vivado’s place and route directives to generate multiple implementation variants of each base design. Specifically, we applied four directive combinations: baseline (default), Explore place + Explore route, ExtraNetDelay_high place + AggressiveExplore route, and Explore place + AggressiveExplore route. These directives explore different optimization trade-offs in placement and routing, resulting in bitstreams with distinct but functionally equivalent implementations.

This augmentation strategy yielded a final dataset of 1,090 bitstreams (approximately $5\times$ the original size) with the following distribution: AES (250), RS232 (250), EthernetMAC10GE (80), BasicRSA (40), PIC16F84 (40), wb_conmax (30), b15 (40), b19 (250), s15850 (10), s35932 (30), s38417 (30), s38584 (30), and vga_lcd (10). Each of these 1,090 bitstreams exists in both benign and Trojan-inserted variants, providing a balanced and diverse dataset suitable for supervised learning while ensuring that augmentation does not artificially bias the detection framework toward artifacts introduced by place/route directives rather than genuine malicious modifications.

V. EXPERIMENTAL EVALUATION

We evaluated the proposed framework on the expanded dataset of 1,090 FPGA bitstreams comprising both benign

configurations and Trojan-inserted variants synthesized for the Xilinx PYNQ-Z1 platform. The dataset is nearly balanced, with 545 benign and 545 malicious samples. To ensure diversity and improve generalization, the dataset spans thirteen design families as detailed in Section 4.

A. Classification Performance

To assess detection effectiveness, we evaluated multiple supervised machine learning models using stratified 5-fold cross-validation on an 80/20 train-test split (872 training samples, 218 test samples). Performance was measured using Accuracy, Precision, Recall, and macro-averaged F1-score.

Table I summarizes the cross-validated results. Among all models, the Random Forest classifier achieved the best overall performance, with an average cross-validation F1-score of 0.88 ± 0.02 . Following hyperparameter tuning via grid search, the optimized Random Forest model achieved a test set accuracy of 0.91 and a macro F1-score of 0.91, indicating strong and balanced detection capability across both benign and malicious classes. Gradient Boosting, AdaBoost, KNN, and Decision Tree classifiers also demonstrated competitive performance, while Naive Bayes showed reduced effectiveness on this expanded feature representation.

TABLE I
CROSS-VALIDATION PERFORMANCE: INITIAL MODEL COMPARISON

Model	Acc.	Prec.	Rec.	F1
RF	0.88 ± 0.02	0.88 ± 0.02	0.88 ± 0.02	0.88 ± 0.02
GB	0.87 ± 0.02	0.88 ± 0.02	0.87 ± 0.02	0.87 ± 0.02
AB	0.87 ± 0.02	0.88 ± 0.02	0.87 ± 0.02	0.87 ± 0.02
LR	0.86 ± 0.02	0.86 ± 0.02	0.86 ± 0.02	0.86 ± 0.02
NB	0.72 ± 0.06	0.78 ± 0.08	0.72 ± 0.06	0.71 ± 0.06
SVM (RBF)	0.83 ± 0.02	0.87 ± 0.01	0.83 ± 0.02	0.82 ± 0.02
KNN	0.87 ± 0.02	0.87 ± 0.02	0.87 ± 0.02	0.87 ± 0.02
DT	0.87 ± 0.02	0.88 ± 0.02	0.87 ± 0.02	0.87 ± 0.02

The tuned Random Forest classifier (300 estimators, max depth 10, min samples split 5, class weight adjusted) achieved a test set precision of 0.92 and recall of 0.91. The confusion matrix revealed 92 true negatives, 106 true positives, 17 false positives, and only 3 false negatives, demonstrating high sensitivity to malicious configurations with a low miss rate.

B. On-Device Inference Performance

To evaluate deployability in embedded environments, the best-performing model (Random Forest) was serialized and deployed on the PYNQ-Z1 platform. We measured inference latency across five representative trials, breaking execution time into three stages: bitstream load time, feature extraction time, and prediction time.

Table II reports the classification results and timing breakdown for each trial. The average end-to-end latency was approximately 16.69 seconds per sample, dominated by feature extraction, which includes reading the bitstream and computing the 278-dimensional feature representation. In contrast, the prediction stage required less than 20 ms per sample,

demonstrating that the trained model itself introduces negligible overhead.

TABLE II
CLASSIFICATION, HARDWARE FAMILY, BENCHMARK, AND TIMING PER TRIAL

Trial	Act. Cls.	Pred. Cls.	Fam.	Bench.	Load (s)	Extr. (s)	Pred. (s)
1	Mal.	Mal.	CRYPTO	AES	0.18	16.56	0.12
2	Ben.	Ben.	CRYPTO	AES	0.02	16.46	0.11
3	Mal.	Mal.	ITC99	b15	0.02	16.41	0.13
4	Ben.	Ben.	COMMS	RS232	0.19	16.44	0.08
5	Ben.	Ben.	CRYPTO	AES	0.02	16.56	0.12

C. Feature Importance and Family Classification

Analysis of feature importance revealed that structural and statistical features provide strong discriminative power. The top-ranked features include log-transformed file size (importance: 0.069), chunk-level mean and standard deviation statistics, specific byte frequencies (bytes 187, 32, 70), nibble balance, zero-byte ratio, and byte transition rate. This confirms that the expanded feature set captures meaningful patterns beyond simple byte histograms.

Additionally, we evaluated the framework's ability to classify bitstreams by design family. Using the same feature representation, a Random Forest classifier achieved perfect classification ($F1 = 1.00$) in all 13 families in the test set, demonstrating that the extracted features encode sufficient structural information to distinguish between different application domains (cryptographic cores, communication interfaces, processors and standard benchmarks).

In general, these findings demonstrate that supervised machine learning applied to an enriched set of bitstream-derived characteristics can effectively detect structured hardware Trojans while also allowing the identification of design families, highlighting the practical utility of the approach for both security screening and bitstream provenance analysis [20], [25].

VI. CONCLUSION AND FUTURE WORK

This work demonstrates that AI-based predictive modeling can provide an effective and deployable mechanism for detecting malicious FPGA configurations directly from bitstreams. By operating solely on binary configuration data, the proposed approach avoids reliance on source code, netlists, or trusted toolchains—artifacts that are often unavailable or untrustworthy in embedded, cloud, and multi-tenant reconfigurable systems. This design prioritizes deployability and autonomy, enabling fast pre-deployment screening on resource-constrained platforms. The framework intentionally trades deep structural interpretability for speed and scalability. Unlike reverse-engineering, side-channel, or graph-based localization methods that aim to identify specific malicious logic elements, the proposed pipeline focuses on rapid detection of anomalous configurations before execution and is best viewed as a lightweight, first-line defense complementary to more detailed

offline analyses [7], [16], [28]. The increasing feasibility of automated Trojan insertion and toolchain compromise further motivates bitstream-only defenses that do not assume trusted intermediate artifacts [15], [18]. Several limitations remain. Because the method relies on global statistical features, extremely small or carefully camouflaged modifications may evade detection. In addition, the current implementation analyzes complete bitstreams prior to deployment and does not yet address partial reconfiguration or continuous runtime monitoring. While the expanded dataset improves diversity, further validation across additional FPGA vendors and device families is needed to strengthen generalization claims.

Future work will focus on expanding evaluation across heterogeneous FPGA platforms, incorporating hybrid statistical and lightweight structural features, and exploring golden-free and unsupervised learning techniques to improve robustness to previously unseen threats [10], [22], [26]. System-level optimizations, including native or FPGA-accelerated feature extraction, could further reduce screening latency and enable near-continuous protection in dynamic reconfiguration scenarios. With the availability of a larger dataset, exploring deep learning architectures also represents a promising direction for enhancing detection capability and generalization.

Overall, this work introduces BLADEI, a practical, AI-driven, bitstream-only security framework that integrates naturally into reconfigurable computing workflows and addresses a critical need for deployable hardware security in modern FPGA-based systems.

CODE AND DATA AVAILABILITY

The BLADEI framework, including source code, feature extraction modules, pre-trained models, and comprehensive documentation, is publicly available at https://github.com/Bread2002/PYNQ_BLADEI under the BSD-3-Clause license. The experimental dataset is available upon request from the corresponding author.

REFERENCES

- [1] C. Maxfield, *The Design Warrior's Guide to FPGAs: Devices, Tools and Flows*. Elsevier, 2004.
- [2] W. Wolf, *FPGA-Based System Design*. Pearson, 2004.
- [3] P. Alfke *et al.*, "It's an FPGA!" *IEEE Solid-State Circuits Magazine*, vol. 3, no. 4, pp. 15–20, 2011.
- [4] R. Karakchi and J. D. Bakos, "Napoly: A non-deterministic automata processor overlay," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 3, pp. 1–25, 2023.
- [5] R. Karakchi, L. O. Richards, and J. D. Bakos, "A dynamically reconfigurable automata processor overlay," in *2017 International Conference on Reconfigurable Computing and FPGAs (ReConFig)*. IEEE, 2017, pp. 1–8.
- [6] R. S. Chakraborty *et al.*, "Hardware trojan insertion by bitstream modification," *IEEE Design & Test*, vol. 30, no. 2, pp. 45–54, 2013.
- [7] F. Benz, A. Seffrin, and S. A. Huss, "Bil: A tool-chain for bitstream reverse-engineering," in *Proceedings of the IEEE International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2012, pp. 735–738.
- [8] J. Boudjadar *et al.*, "Dynamic fpga reconfiguration for embedded ai," *Future Generation Computer Systems*, vol. 169, p. 107777, 2025.
- [9] J. Hou *et al.*, "Hardware trojan attacks on fpga-based cnn accelerators," *Micromachines*, vol. 15, no. 1, p. 149, 2024.
- [10] J. Zhou *et al.*, "Security of sram-based fpgas in the era of ai," *Journal of Low Power Electronics and Applications*, vol. 15, no. 4, p. 66, 2025.
- [11] AMD, "PYNQ™: Python Productivity for Zynq," <https://www.pynq.io>, 2024, accessed: Aug. 4, 2025.
- [12] S. Mal-Sarkar *et al.*, "Design and validation for fpga trust," *IEEE Transactions on Multi-Scale Computing Systems*, vol. 2, no. 3, pp. 186–198, 2016.
- [13] C. Marchand and J. Francq, "Stealthy hardware trojans on fpgas," *IET Computers & Digital Techniques*, vol. 8, no. 6, pp. 246–255, 2014.
- [14] M. K. Ahmed *et al.*, "Multi-tenant cloud fpga: Security, trust, and privacy," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 2, 2025.
- [15] C. Krieg, C. Wolf, and A. Jantsch, "Malicious lut: Stealthy fpga trojan," in *ICCAD*, 2016.
- [16] J. Yoon, Y. Seo, J. Jang, M. Cho, J. Kim, H. Kim, and T. Kwon, "A bitstream reverse engineering tool for fpga hardware trojan detection," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 2318–2320. [Online]. Available: <https://doi.org/10.1145/3243734.3278487>
- [17] Y. Seo, J. Yoon, J. Jang, M. Cho, H.-K. Kim, and T. Kwon, "Poster: Towards reverse engineering fpga bitstreams for hardware trojan detection," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2018, pp. 18–21.
- [18] V. R. Surabhi *et al.*, "Feint: Automated trojan insertion framework," *Information*, vol. 15, no. 7, p. 395, 2024.
- [19] C. Krieg, "Reflections on trusting trusthub," in *ICCAD*, 2023.
- [20] R. Elnaggar *et al.*, "Learning malicious circuits in fpga bitstreams," *IEEE TCAD*, vol. 42, no. 3, pp. 726–739, 2022.
- [21] R. Elnaggar and K. Chakraborty, "Machine learning for hardware security: Opportunities and risks," *Journal of Electronic Testing*, vol. 34, no. 2, pp. 183–201, 2018.
- [22] A. Ghimire *et al.*, "Golden-free unsupervised ml for trojan detection," *JETC*, vol. 21, no. 3, 2025.
- [23] D. M. Shila and V. Venugopal, "Security analysis of hardware trojan threats in fpga," in *IEEE ICC*, 2014.
- [24] K. S. Kumar *et al.*, "Improved aes hardware trojan benchmark," in *VLSI Design and Test*, 2015.
- [25] R. Elnaggar, J. Chaudhuri, R. Karri, and K. Chakraborty, "Learning malicious circuits in fpga bitstreams," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 3, pp. 726–739, 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/9828544/>
- [26] V. More *et al.*, "Nlp meets hardware trojan detection," in *ISVLSI*, 2024.
- [27] J. Dofe *et al.*, "Nlp for hardware trojan detection in fpgas," *Cryptography*, vol. 8, no. 3, p. 36, 2024.
- [28] H. Su *et al.*, "Explainable hardware trojan localization at lut level," *IEEE TCAD*, 2025.
- [29] V. T. Hayashi and W. V. Ruggiero, "Hardware trojan detection in open-source hardware designs using machine learning," *IEEE Access*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10904479/>
- [30] Scikit-learn Developers, "Truncated svd," <https://bit.ly/4mmi4BT>, 2025, accessed: Aug. 4, 2025.
- [31] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://dl.acm.org/doi/10.5555/1953048.2078195>
- [32] R. Stahle-Smith and R. Karakchi, "Real-time ml-based defense against malicious payload," *arXiv preprint arXiv:2509.02387*, 2025.
- [33] N. Chawla, K. Bowyer, L. Hall, and W. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [34] Imbalanced-learn Developers, "Smote," <https://bit.ly/3IXc0l7>, 2024, accessed: Aug. 4, 2025.
- [35] Scikit-learn Developers, "Cross validation," <https://bit.ly/3get8QG>, 2025, accessed: Aug. 4, 2025.
- [36] Bread2002, "PYNQ_BLADEI," 2025, GitHub Repository.
- [37] N. Chinnasami, R. S. Smith, and R. Karakchi, "Poster: Hybrid monitoring for side-channel security in edge socs," in *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing*, 2025, pp. 1–4.
- [38] N. Chinnasami and R. Karakchi, "Hybrid cryptographic monitoring system for side-channel attack detection on pynq socs," *arXiv preprint arXiv:2508.21606*, 2025.